

Universidad Mayor de San Andrés

Facultad de Ciencias Puras y Naturales

Carrera de Informática



El Observador Digital

Plataforma de análisis comparativo
de medios bolivianos

Maximiliano Gómez Mallo

14480221

Materia: Minería de Datos

Docente: Menfy Morales Rios

Fecha: La Paz, Bolivia — Mayo 2026

Índice

1	Resumen Ejecutivo	3
2	Motivación	3
3	Arquitectura General	4
4	Recolección de Datos mediante Web Scraping	4
4.1	El problema del RSS en medios bolivianos	4
4.2	Arquitectura del pipeline de scraping	5
4.3	Configuración por medio: patrones de URL	6
4.4	HTTP robusto: reintentos con backoff exponencial	7
4.5	Extracción del cuerpo y la fecha del artículo	8
4.5.1	Selectores CSS en cascada	8
4.5.2	Extracción de la fecha de publicación	9
4.6	Extracción de la imagen de portada (og:image)	10
4.7	Filtros de calidad	10
4.8	Orquestación y ejecución periódica	11
4.9	Manejo de zonas horarias	12
5	Agrupación de Eventos con IA Local	12
5.1	Representación vectorial de artículos (<i>embeddings</i>)	12
5.2	Similitud coseno	13
5.3	Algoritmo Union-Find para clustering	13
5.4	Criterio de creación de un evento: cobertura cruzada obligatoria	14
5.5	Incorporación de artículos nuevos a eventos existentes	14
5.6	Extracción de palabras clave con KeyBERT	15
6	Análisis Editorial con Gemini	15
6.1	Análisis de tono por artículo	15
6.2	Análisis de sesgo comparativo entre medios	16
6.3	Gestión de límites de la API gratuita	16
7	API REST (FastAPI)	16
8	Frontend (Astro + React)	18
8.1	Portada (/)	19
8.2	Cronología (/cronologia)	20
8.3	Sesgos (/sesgos)	20
8.4	Stack tecnológico	21

9 Decisiones de Diseño	21
10 Despliegue	21
11 Resultados y Ejemplos	22
12 Limitaciones y Trabajo Futuro	22

1. Resumen Ejecutivo

El Observador Digital es una plataforma automática de monitoreo y análisis comparativo de medios de comunicación bolivianos. El sistema vigila seis portales de noticias — Red Uno, El Deber, Brújula Digital, Los Tiempos, Erbol y La Razón — recolectando artículos nuevos **una vez al día** y procesándolos con técnicas de inteligencia artificial local y en la nube.

El flujo de trabajo central es el siguiente: los artículos recién descargados se convierten en vectores numéricos mediante un modelo de lenguaje local (*MiniLM-L12-v2*), se calcula la similitud entre ellos y se agrupan automáticamente aquellos que tratan el mismo evento noticioso. Solo se forma un evento si al menos **dos medios distintos** cubren el mismo hecho — esto garantiza que el sistema detecta únicamente noticias con cobertura cruzada real, descartando artículos aislados. Posteriormente, la API de Gemini analiza el tono editorial de cada artículo y detecta diferencias de encuadre entre los distintos medios que cubren el mismo hecho.

Los resultados son accesibles a través de una interfaz web con estética de periódico clásico: portada con tarjetas visuales (incluida la imagen de portada de cada evento), cronología de eventos con gráfico de dispersión por sesgo e importancia, y visualización detallada de sesgos editoriales. Adicionalmente, el sistema ofrece un asistente de inteligencia artificial conversacional flotante que responde preguntas sobre los datos monitoreados en lenguaje natural y devuelve tarjetas con los eventos o artículos más relevantes para cada consulta.

La plataforma corre en un servidor VPS con costes de infraestructura mínimos: el modelo de *embeddings* funciona en CPU sin costo alguno, y la API de Gemini se usa en el nivel gratuito.

2. Motivación

Bolivia tiene un ecosistema mediático marcadamente fragmentado. Los principales diarios y portales de noticias se concentran en tres ciudades — Santa Cruz de la Sierra, Cochabamba y La Paz — y cada uno responde a una línea editorial propia, condicionada por factores regionales, económicos y políticos. Este fenómeno es conocido en el ámbito del periodismo como *sesgo editorial*: la tendencia de un medio a presentar un mismo hecho desde un encuadre particular, seleccionando ciertos datos, omitiendo otros y eligiendo un vocabulario con carga valorativa.

Un ejemplo cotidiano: una manifestación en La Paz puede aparecer en un portal paceño como “protesta ciudadana legítima” y en un diario cruceño como “bloqueo que afecta la economía”. Ambos describen el mismo evento; ninguno miente; pero la percepción que transmiten al lector es radicalmente distinta.

Monitorear manualmente estos sesgos es inviable. Un analista tendría que leer decenas de artículos al día de múltiples portales, identificar cuáles tratan el mismo tema, compararlos y registrar las diferencias. El tiempo necesario para esa tarea supera con creces los recursos de cualquier investigador individual.

El Observador Digital automatiza ese proceso completo. En lugar de requerir intervención humana para agrupar artículos o detectar diferencias editoriales, el sistema:

- Descarga artículos de seis portales de forma continua y estructurada.
- Identifica automáticamente qué artículos de distintos medios hablan del mismo evento, sin necesidad de que compartan palabras clave exactas.
- Cuantifica y visualiza las diferencias de encuadre entre medios para el mismo evento.
- Proporciona una interfaz de consulta en lenguaje natural para que cualquier usuario pueda explorar los datos.

El objetivo no es juzgar qué medio es más o menos confiable, sino ofrecer una herramienta transparente que permita al lector ver el panorama completo de cómo se está cubriendo un tema en Bolivia.

3. Arquitectura General

El sistema está organizado en cuatro capas funcionales que operan como un *pipeline* secuencial. La Figura 1 muestra el flujo de datos desde la recolección hasta la presentación.

El **programador de tareas** (*APScheduler*) coordina dos ciclos diarios: recolección a las 10:00 UTC y análisis a las 11:00 UTC. El ciclo de recolección descarga artículos nuevos y sus imágenes de portada; el ciclo de análisis agrupa los artículos en eventos y ejecuta el análisis editorial con Gemini. Ambos ciclos son independientes: el análisis siempre trabaja sobre los artículos ya almacenados en la base de datos.

4. Recolección de Datos mediante Web Scraping

El web scraping es la técnica central de este proyecto. Dado que los medios bolivianos no ofrecen APIs públicas ni feeds confiables, el sistema extrae información directamente del HTML de sus portales de forma automatizada. Esta sección describe en detalle la arquitectura, las estrategias por medio, el manejo de errores HTTP y los algoritmos de extracción de contenido.

4.1. El problema del RSS en medios bolivianos

La sindicación RSS es el mecanismo estándar que los portales exponen para que agregadores reciban contenido nuevo. En teoría, basta con suscribirse al feed de cada

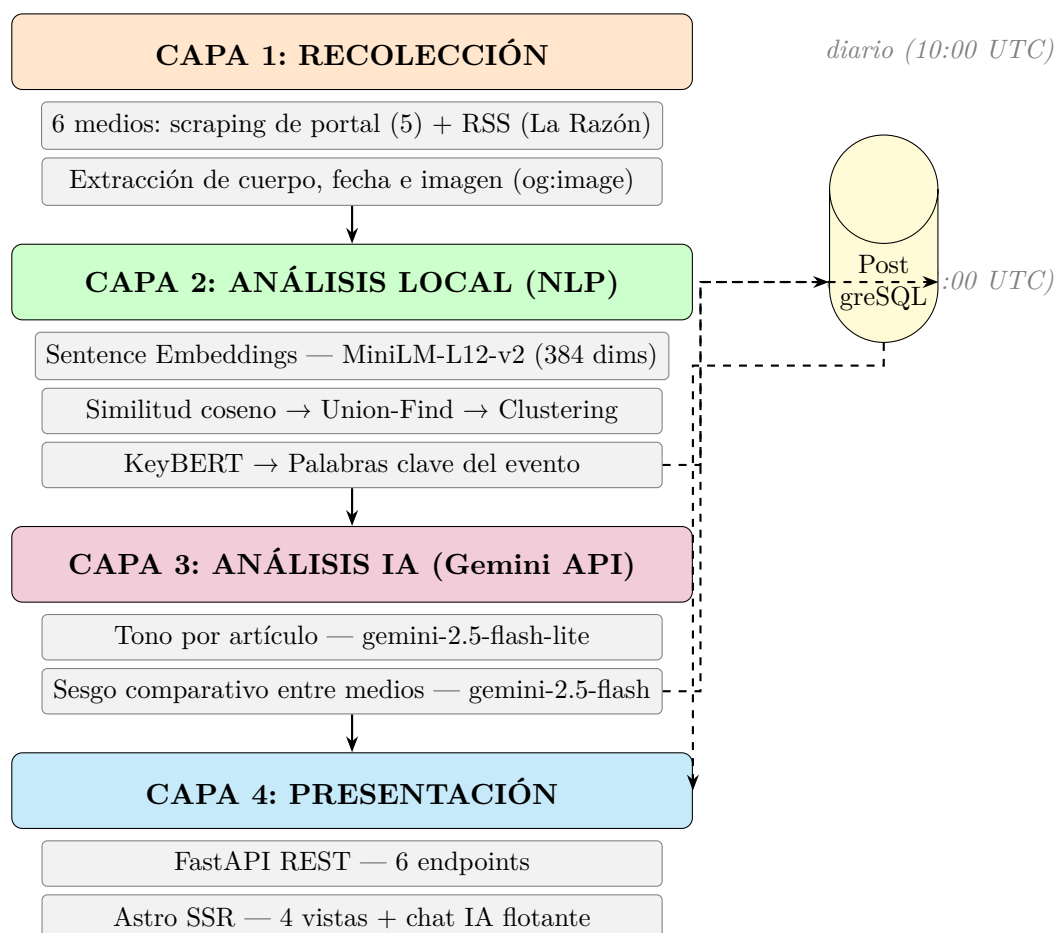


Figura 1: Pipeline de cuatro capas del sistema El Observador Digital.

medio. En la práctica boliviana ese supuesto falla por dos razones:

- Varios portales relevantes no ofrecen feed RSS o lo tienen deshabilitado. Este es un problema común en países donde el mantenimiento técnico de los portales de noticias es esporádico y los equipos de desarrollo son reducidos.
- Los feeds que sí existen se actualizan con un retraso de dos a tres semanas: el equipo publica primero en la web y el RSS se actualiza de forma manual o nunca.

La solución adoptada es un enfoque **híbrido**: scraping directo del portal para cinco medios (Red Uno, El Deber, Brújula Digital, Los Tiempos, Erbol) y RSS únicamente para La Razón, cuyo sitio es una *Single Page Application* (SPA) renderizada con JavaScript que hace imposible el scraping tradicional del DOM.

4.2. Arquitectura del pipeline de scraping

El proceso se divide en dos etapas secuenciales dentro del ciclo diario de recolección:

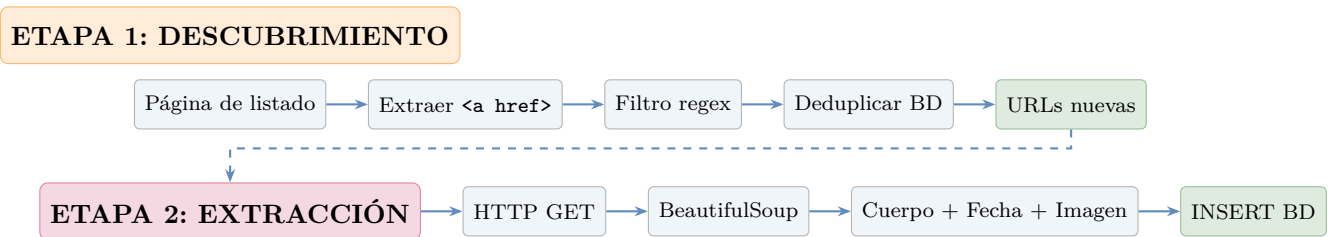


Figura 2: Pipeline de scraping en dos etapas: descubrimiento de URLs y extracción de contenido.

La **Etapa 1** (descubrimiento) descarga la portada de cada medio, extrae todos los hipervínculos y aplica un filtro por expresión regular para quedarse solo con URLs de artículos. La **Etapa 2** (extracción) visita cada URL nueva y extrae el cuerpo textual, la fecha de publicación y la imagen de portada.

4.3. Configuración por medio: patrones de URL

Cada portal tiene una estructura de URLs distinta. La Tabla 1 resume la estrategia y el patrón regex usado para cada medio.

Cuadro 1: Estrategia de scraping y patrón de URL por medio

Medio	Estrategia	Patrón regex	Característica
Red Uno	Portal	<code>/(noticias ...)/\S+\d{6,}</code>	≥ 6 dígitos en path
El Deber	Portal	<code>^[a-z-]+/[a-z0-9-]{10,}_</code>	Slug con <code>_</code> al final
Brújula Digital	Portal	<code>^[a-z-]+/[a-z0-9-]{10,}</code>	Slug ≥ 10 caracteres
Los Tiempos	Portal	<code>/\d{8}/</code>	8 dígitos consecutivos
Erbol	Portal	<code>^[^/\s]+/[^\s]{10,}\$</code>	Acepta chars URL-encoded
La Razón	RSS	Feed XML (feedparser)	SPA React, sin DOM estático

El diccionario `CONFIGS` en `portal.py` centraliza esta configuración. Cada entrada define la URL de listado, la URL base para normalizar href relativos, el patrón regex de artículo y una lista de fragmentos a excluir (páginas de tags, autores, búsquedas):

```

1 CONFIGS = {
2     "El Deber": {
3         "listing_url": "https://eldeber.com.bo",
4         "base_url": "https://eldeber.com.bo",
5         "url_pattern": r"^[a-z-]+/[a-z0-9-]{10,}_",
  
```

```
6     "exclude_fragments": ["/tag/", "/autor/", "/seccion/"],
7 },
8 "Los Tiempos": {
9     "listing_url": "https://www.lostiempos.com",
10    "base_url":     "https://www.lostiempos.com",
11    "url_pattern":  r"/\d{8}/",
12    "exclude_fragments": [],
13 },
14 # ... Red Uno, Erbol, Brújula Digital
15 }
```

Listing 1: Configuración de scraping por medio (portal.py)

4.4. HTTP robusto: reintentos con backoff exponencial

Los portales bolivianos presentan inestabilidad frecuente: Brújula Digital y Erbol retornan HTTP 500 en una fracción de artículos, y Red Uno ocasionalmente limita la velocidad de acceso. La función `_fetch()` implementa reintentos con **espera exponencial** para manejar estos fallos de forma transparente:

```
1 HEADERS = {
2     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
3                 "AppleWebKit/537.36 Chrome/124.0 Safari/537.36",
4     "Accept": "text/html,application/xhtml+xml,*/*;q=0.8",
5     "Accept-Language": "es-ES,es;q=0.9",
6 }
7
8 def _fetch(url, reintentos=3):
9     for i in range(reintentos):
10         try:
11             r = requests.get(url, headers=HEADERS, timeout=15)
12             if r.status_code == 200:
13                 return r.text
14             logger.warning(f"HTTP {r.status_code} en {url}")
15         except Exception as e:
16             logger.warning(f"Intento {i+1} fallido para {url}: {e}")
17         if i < reintentos - 1:
18             time.sleep(2 ** i) # 1s, 2s entre intentos
19     return None
```

Listing 2: Función `_fetch` con reintentos y backoff exponencial (portal.py)

Aspectos relevantes de la implementación:

- **User-Agent realista:** simula un navegador Chrome para evitar bloqueos por bots. Varios portales bolivianos rechazan peticiones con `python-requests` como User-Agent.

- **Timeout de 15 segundos:** evita que una conexión colgada bloquee el ciclo completo de scraping.
- **Backoff exponencial:** espera $2^0 = 1$ segundo tras el primer fallo y $2^1 = 2$ segundos tras el segundo, reduciendo la carga sobre el servidor en momentos de alta demanda.
- **Fallo silencioso:** si los 3 intentos fallan, retorna `None` y el artículo queda pendiente para el siguiente ciclo diario.

Una vez obtenido el HTML, la extracción de URLs de artículos filtra en dos niveles: primero aplica el patrón regex del medio, luego verifica que ningún fragmento excluido esté presente en el path. Los títulos con menos de 10 caracteres (links de iconos o imágenes sin texto) también se descartan.

4.5. Extracción del cuerpo y la fecha del artículo

La heterogeneidad de los portales bolivianos — algunos usan WordPress, otros sistemas propietarios, otros frameworks modernos — obliga a una estrategia de extracción tolerante a la variación de estructura HTML.

4.5.1. Selectores CSS en cascada

`article.py` define una lista ordenada de 8 selectores CSS que se prueban secuencialmente. Se usa el primero que devuelva un bloque de texto con al menos 200 caracteres:

Cuadro 2: Selectores CSS en orden de prioridad para extracción del cuerpo

#	Selector	Justificación
1	<code>article</code>	Elemento HTML5 semántico; estándar en CMS modernos
2	<code>[class*='article-body']</code>	Clase más usada en portales de noticias personalizados
3	<code>[class*='article_body']</code>	Variante con guión bajo
4	<code>[class*='content-body']</code>	Patrón alternativo
5	<code>[class*='nota-body']</code>	Terminología de medios hispanohablantes
6	<code>[class*='cuerpo']</code>	Término en español, común en portales bolivianos
7	<code>.entry-content</code>	Clase estándar de WordPress
8	<code>main</code>	Fallback final: elemento HTML5 main

Antes de aplicar los selectores, se elimina el “ruido” del HTML: scripts, estilos, navegación, pie de página, sidebars y elementos publicitarios. Esto garantiza que el texto extraído sea solo el contenido editorial:

```
1 SELECTORES = ["article", "[class*='article-body']",
```

```

2         "[class*='article_body']", "[class*='content-body']",
3         "[class*='nota-body']", "[class*='cuerpo']",
4         ".entry-content", "main"]
5
6 RUIDO = ["script", "style", "nav", "footer", "aside", "[class*='ad']"]
7
8 soup = BeautifulSoup(r.text, "lxml")
9
10 for selector in RUIDO:           # eliminar elementos no editoriales
11     for tag in soup.select(selector):
12         tag.decompose()
13
14 cuerpo = None
15 for selector in SELECTORES:      # probar selectores en cascada
16     elemento = soup.select_one(selector)
17     if elemento:
18         texto = elemento.get_text(separator=" ", strip=True)
19         if len(texto) >= 200:     # umbral de contenido minimo
20             cuerpo = texto
21             break

```

Listing 3: Extracción robusta del cuerpo (article.py)

4.5.2. Extracción de la fecha de publicación

La fecha de publicación se extrae con tres métodos en orden de fiabilidad decreciente:

```

1 def _extraer_fecha(soup):
2     # 1. OpenGraph: mas estandarizado
3     meta = soup.find("meta", property="article:published_time")
4     if meta and meta.get("content"):
5         f = _parse_fecha(meta["content"])
6         if f: return f
7
8     # 2. JSON-LD: datos estructurados embebidos
9     for script in soup.find_all("script", type="application/ld+json"):
10         try:
11             data = json.loads(script.string or "")
12             if isinstance(data, list): data = data[0]
13             fecha_str = data.get("datePublished") or data.get("dateCreated")
14             if fecha_str:
15                 f = _parse_fecha(fecha_str)
16                 if f: return f
17         except Exception: pass
18
19     # 3. <time datetime>: fallback HTML5

```

```
20     time_tag = soup.find("time", attrs={"datetime": True})
21     if time_tag:
22         return _parse_fecha(time_tag["datetime"])
23     return None
```

Listing 4: Extracción de fecha en tres niveles de prioridad (article.py)

4.6. Extracción de la imagen de portada (og:image)

Los portales de noticias modernos publican una imagen destacada para cada artículo con el fin de que se muestre correctamente al compartir en redes sociales. Esta imagen se declara en el encabezado HTML como metadato **Open Graph** (og:image), un estándar ampliamente adoptado por Facebook, Twitter y WhatsApp.

El Observador Digital aprovecha este metadato para mostrar la imagen de portada de cada evento sin necesidad de alojar imágenes propias: simplemente se lee la URL que el propio portal ya declaró y se presenta en la interfaz.

```
1 def _extraer_imagen(soup, base_url):
2     """Retorna la URL absoluta de og:image, o None si no existe."""
3     og = (soup.find("meta", property="og:image")
4           or soup.find("meta", attrs={"name": "og:image"}))
5     if not og:
6         return None
7     url = (og.get("content") or "").strip()
8     if not url:
9         return None
10    # Convertir URL relativa a absoluta si es necesario
11    if url.startswith("//"):
12        return "https:" + url
13    if url.startswith("/"):
14        return base_url.rstrip("/") + url
15    return url
```

Listing 5: Extracción de imagen og:image (article.py)

La URL obtenida se almacena en la columna `imagen_url` de la tabla `articulos` (añadida mediante una migración de Alembic). Cuando la API devuelve un evento, selecciona la primera imagen no nula entre sus artículos y la expone como `imagen_url` del evento. En el frontend, si la imagen no carga (error HTTP, URL expirada), el componente muestra una franja de color de la categoría del evento como alternativa visual.

4.7. Filtros de calidad

Antes de insertar un artículo en la base de datos se aplican cuatro filtros que garantizan la integridad del corpus:

- **Deduplicación por URL:** la tabla tiene una restricción `UNIQUE` sobre la columna `url`; si ya existe, la inserción se omite sin error.
- **Título mínimo:** el título debe tener al menos 10 caracteres (elimina links de iconos o imágenes sin texto).
- **Cuerpo mínimo:** el cuerpo debe superar los 200 caracteres (descarta artículos de solo imagen, video o galería sin texto editorial).
- **Antigüedad máxima:** artículos con fecha de publicación superior a 30 días se descartan al inicio del sistema para evitar poblar la base de datos con contenido histórico irrelevante.

4.8. Orquestación y ejecución periódica

La función `correr_scraping()` en `app/scrapper/__init__.py` orquesta el pipeline completo en dos fases:

```
1 def correr_scraping(db):
2     stats = {"nuevos": 0, "cuerpos": 0, "errores": 0}
3     medios = db.query(Medio).filter(Medio.activo == True).all()
4
5     # Fase 1: descubrir nuevas URLs por medio
6     for medio in medios:
7         if medio.nombre in PORTAL_CONFIGS:
8             stats["nuevos"] += scrape_portal(medio, db) # scraping
9             HTML
10        else:
11            stats["nuevos"] += fetch_feed(medio, db)      # RSS
12            feedparser
13
14    # Fase 2: extraer cuerpo, fecha e imagen de articulos sin
15    completar
16    sin_cuerpo = db.query(Articulo).filter(Articulo.cuerpo == None).
17    all()
18    for articulo in sin_cuerpo:
19        resultado = scrape_articulo(articulo.url)
20        if resultado["cuerpo"]:
21            articulo.cuerpo = resultado["cuerpo"]
22            stats["cuerpos"] += 1
23        if resultado["fecha_publicacion"] and articulo.
24            fecha_publicacion is None:
25            articulo.fecha_publicacion = resultado["fecha_publicacion"]
26        ]
27        if resultado.get("imagen_url") and articulo.imagen_url is None
28        :
29            articulo.imagen_url = resultado["imagen_url"]
30
31    db.commit()
```

```
25 logger.info(f"Scraping completo: {stats}")
26 return stats
```

Listing 6: Función de orquestación del scraping (`__init__.py`)

Esta función es invocada por **APScheduler** con un disparador *cron* a las 10:00 UTC cada día. La separación en dos fases es intencional: si un artículo nuevo se descubre en la Fase 1 pero falla la extracción (timeout, error HTTP), queda registrado en la BD con `cuerpo = NULL` y será reintentado en el siguiente ciclo diario.

4.9. Manejo de zonas horarias

Los portales bolivianos publican las fechas en hora local (UTC-4, sin horario de verano). La base de datos almacena estas fechas *sin información de zona horaria* (*naive datetime*) tal como vienen. La API REST aplica el sufijo adecuado al serializar:

- `fecha_publicacion`: se añade el sufijo `-04:00` (hora boliviana).
- *Timestamps* del sistema (fecha de scraping, detección de eventos): se añade `Z` (UTC).

5. Agrupación de Eventos con IA Local

5.1. Representación vectorial de artículos (*embeddings*)

El primer desafío para agrupar artículos es representarlos de una forma que la computadora pueda comparar. El texto en crudo no es directamente comparable: dos artículos sobre la misma huelga pueden no compartir ninguna palabra si uno usa “paro” y el otro “huelga”.

La solución son los **embeddings semánticos**: una técnica que convierte cualquier texto en una lista de 384 números reales. La metáfora más útil es pensar en esos números como *coordenadas GPS del significado*: textos sobre temas similares terminan cerca en ese espacio de 384 dimensiones, aunque usen vocabulario distinto.

El modelo utilizado es **MiniLM-L12-v2**, parte de la familia *sentence-transformers*. Ocupa aproximadamente 120 MB, se descarga una sola vez y funciona íntegramente en CPU sin necesidad de GPU ni de llamadas a APIs externas. El coste computacional por artículo es de decenas de milisegundos.

```
1 from sentence_transformers import SentenceTransformer
2
3 model = SentenceTransformer(
4     "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
5 )
6
7 # texto = titulo + primeros 512 tokens del cuerpo
8 vector = model.encode(texto) # -> array de 384 floats
```

Listing 7: Generación de embeddings con sentence-transformers

5.2. Similitud coseno

Una vez que cada artículo tiene su vector, el siguiente paso es medir qué tan “parecidos” son dos artículos. La métrica usada es la **similitud coseno**:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

Esta fórmula calcula el coseno del ángulo entre dos vectores. Su rango es de -1 a 1 ; en la práctica, para vectores de texto generados por modelos modernos el rango efectivo es de 0 a 1 :

- 1.0 : artículos idénticos en significado.
- 0.65 – 0.85 : artículos sobre el mismo evento con redacción diferente.
- < 0.4 : artículos sobre temas no relacionados.

El umbral elegido para declarar que dos artículos pertenecen al mismo evento es **0.65**. Este valor se determinó empíricamente: con 0.75 se perdían artículos de distintos medios que cubrían el mismo hecho con vocabulario muy diferente; con 0.65 se capturan esos casos sin agrupar artículos no relacionados.

5.3. Algoritmo Union-Find para clustering

Con las similitudes calculadas, el problema se convierte en: *¿cómo agrupar artículos en eventos?*. El algoritmo **Union-Find** (también llamado *Disjoint Set Union*) resuelve esto eficientemente.

La idea es sencilla:

1. Al inicio, cada artículo está en su propio grupo (evento) individual.
2. Por cada par de artículos (i, j) con similitud > 0.65 : fusionar sus grupos.
3. Al finalizar, todos los artículos en el mismo grupo conforman un evento.

```
1 parent = {i: i for i in articulos}
2
3 def find(x):
4     if parent[x] != x:
5         parent[x] = find(parent[x]) # compresion de camino
6     return parent[x]
7
8 def union(x, y):
9     parent[find(x)] = find(y)
10
11 for i in articulos:
12     for j in articulos:
```

```
13     if i != j and similitud(i, j) > 0.65:
14         union(i, j)
15
16 # Resultado: grupos de IDs por find(id)
```

Listing 8: Pseudocódigo simplificado de Union-Find

El resultado es un conjunto de *clusters*: cada cluster contiene los artículos que el algoritmo considera que tratan el mismo evento noticioso, independientemente del medio que los publicó.

5.4. Criterio de creación de un evento: cobertura cruzada obligatoria

No todo cluster se convierte en un evento. El sistema aplica un filtro fundamental: **un cluster solo genera un evento si contiene artículos de dos o más medios distintos**.

¿Por qué este requisito? Porque el objetivo del sistema es analizar *diferencias de cobertura entre medios*. Un artículo publicado por un único portal no permite comparación alguna; no tiene sentido almacenarlo, analizarlo con Gemini o mostrarlo al usuario como “evento”. Este filtro elimina el ruido de artículos aislados y garantiza que cada evento en la base de datos representa una noticia con relevancia suficiente como para haber llamado la atención de múltiples redacciones.

```
1 medios_en_cluster = {art.medio_id for art in cluster}
2 if len(medios_en_cluster) < 2:
3     continue # cluster de un solo medio -> descartar
4 # Crear evento solo si hay >= 2 medios distintos
5 nuevo_evento = Evento(titulo=..., score_importancia=len(
    medios_en_cluster)/6)
```

Listing 9: Filtro de cobertura cruzada en embeddings.py

5.5. Incorporación de artículos nuevos a eventos existentes

Los eventos noticiosos no terminan en un día. Una huelga que comienza el lunes puede seguir generando artículos el martes y el miércoles. El sistema maneja este escenario comparando los artículos nuevos del día con los eventos ya existentes en la base de datos.

El proceso funciona así:

1. Se selecciona un artículo representativo de cada evento de los últimos **7 días** y se calcula su embedding.
2. Para cada cluster de artículos nuevos, se calcula el **centro del cluster** (promedio de sus embeddings).
3. Se compara el centro del cluster con el embedding de cada evento existente.

4. Si la similitud supera 0.65: los artículos nuevos se asignan al evento existente en lugar de crear uno nuevo.

Esto evita que el mismo evento aparezca duplicado en múltiples entradas y permite que el sistema “acumule” la cobertura de un tema a lo largo de varios días de seguimiento.

5.6. Extracción de palabras clave con KeyBERT

Una vez formado un cluster, el sistema extrae las cinco palabras o frases más representativas del evento usando **KeyBERT**. Este algoritmo utiliza internamente BERT para identificar los fragmentos de texto cuyo embedding sea más similar al embedding del documento completo, es decir, las frases que mejor “resumen” el contenido.

La entrada es la concatenación de los títulos y cuerpos de todos los artículos del cluster. La salida es una lista de hasta cinco frases clave:

```
1 ["reforma tributaria", "gobierno", "impuesto", "asambleistas", "decreto"]
```

Listing 10: Ejemplo de salida de KeyBERT

Estas palabras clave se almacenan en el campo **temas** del evento y se usan para filtrado, categorización y presentación en el frontend.

6. Análisis Editorial con Gemini

6.1. Análisis de tono por artículo

Una vez que un artículo tiene cuerpo de texto **y pertenece a un evento**, se envía a **gemini-2.5-flash-lite** para determinar su tono editorial. El requisito de pertenecer a un evento es importante: no tiene sentido gastar llamadas a la API para analizar artículos que no van a participar en ninguna comparación de sesgos.

Este modelo más pequeño y económico es suficiente para un análisis por artículo individual. La petición incluye el título y los primeros 1500 caracteres del cuerpo, con temperatura 0.1 (valores bajos hacen la respuesta más determinista y reproducible). El *prompt* solicita una respuesta en formato JSON:

```
1 {  
2   "tono": "positivo | negativo | neutral",  
3   "temas": ["tema1", "tema2"],  
4   "fuentes_citadas": ["fuente1", "fuente2"]  
5 }
```

Listing 11: Esquema de respuesta de análisis de tono

Si el artículo no tiene texto (solo imagen o video), el campo **analisis** se marca como **sin_texto** y no se realiza la llamada a la API. El ciclo de análisis procesa hasta **50 artículos por día** con este filtro aplicado.

6.2. Análisis de sesgo comparativo entre medios

El análisis de sesgo requiere al menos dos artículos de **medios distintos** cubriendo el mismo evento, ambos con cuerpo de texto. Se usa **gemini-2.5-flash** (modelo más capaz) para esta tarea de mayor complejidad.

Se envían todos los artículos del evento juntos en una sola petición. El modelo responde con un valor de sesgo por medio, en una escala de -1.0 a $+1.0$:

- -1.0 : encuadre muy negativo / crítico hacia el tema del evento.
- 0.0 : cobertura neutral o equilibrada.
- $+1.0$: encuadre muy positivo / favorable.

La **divergencia** del evento se calcula como la diferencia entre el sesgo máximo y el mínimo entre todos los medios:

$$\text{divergencia} = \max(\text{sesgos}) - \min(\text{sesgos}) \quad (2)$$

Ejemplo real: para un evento sobre política económica, Brújula Digital obtuvo -0.4 (encuadre crítico) y Los Tiempos $+0.2$ (ligeramente favorable), resultando en una divergencia de 0.6 .

6.3. Gestión de límites de la API gratuita

El nivel gratuito de Gemini impone restricciones. Para no agotar la cuota, el ciclo de análisis aplica las siguientes estrategias:

- **Filtro por evento**: solo se analizan artículos que ya pertenecen a un evento (con `evento_id` asignado). Los artículos huérfanos no se envían a la API.
- **Límite diario**: el ciclo procesa máximo 50 artículos para análisis de tono.
- **Pausa entre llamadas**: se introduce una pausa de 4 segundos entre llamadas consecutivas para evitar superar el límite por minuto de la API.

Al limitar el análisis solo a artículos en eventos y ejecutar el ciclo una vez al día en lugar de cada hora, el sistema hace un uso mucho más eficiente de la cuota gratuita y evita los errores de rate limiting que afectaban al sistema anterior.

7. API REST (FastAPI)

El backend expone una API REST construida con **FastAPI**. Todos los endpoints devuelven JSON con soporte CORS habilitado para el frontend.

El parámetro `orden` en `/noticias` acepta `fecha` (por fecha de publicación) o `scraping` (por fecha de recolección, útil para ver lo más reciente). El `score_importancia` de un evento se calcula como la fracción de medios monitoreados que lo cubren: $4/6 = 0.67$.

Cuadro 3: Endpoints principales de la API REST

Método	Endpoint	Parámetros clave	Descripción
GET	/noticias	limit, medio_nombre, orden	Artículos paginados
GET	/eventos	limit	Eventos por importancia
GET	/eventos/{id}	—	Detalle con cobertura por medio
GET	/sesgos	limit	Eventos con mayor divergencia
GET	/medios	—	Lista de medios monitoreados
POST	/chat	mensaje, historial	Asistente IA conversacional

Ejemplo de respuesta de /eventos/{id}

```
1 {
2   "id": 123,
3   "titulo": "Bolivia anuncia reforma tributaria",
4   "fecha_deteccion": "2026-05-14T14:00:00Z",
5   "score_importancia": 0.67,
6   "imagen_url": "https://eldeber.com.bo/img/reforma.jpg",
7   "temas": ["reforma tributaria", "impuesto", "gobierno"],
8   "articulos_por_medio": {
9     "El Deber": [{
10       "titulo": "...",
11       "imagen_url": "https://eldeber.com.bo/img/reforma.jpg",
12       "analisis": {"tono": "negativo", "sesgo": -0.3}
13     }],
14     "Los Tiempos": [{
15       "titulo": "...",
16       "imagen_url": null,
17       "analisis": {"tono": "neutral", "sesgo": 0.1}
18     }]
19   }
20 }
```

Listing 12: Respuesta abreviada del endpoint de detalle de evento

Asistente IA con *function calling* y tarjetas de contenido

El endpoint POST /chat integra Gemini con *function calling* (herramientas). El modelo decide autónomamente cuándo llamar cada herramienta en función de la pregunta del usuario:

- buscar_noticias: busca artículos por texto libre o medio.
- get_eventos: obtiene los eventos más recientes o importantes.
- comparar_cobertura: compara cómo distintos medios cubrieron un evento.
- resumen_estadisticas: devuelve métricas globales del sistema.

Gemini puede encadenar hasta 3 rondas de uso de herramientas antes de generar la respuesta final. Además del texto de respuesta, el endpoint devuelve una lista de **tarjetas** (cards): objetos estructurados que el frontend puede renderizar como componentes visuales navegables. Hay dos tipos de tarjeta:

Cuadro 4: Tipos de tarjeta devueltos por el endpoint de chat

Tipo	Campos clave	Comportamiento en UI
evento	id, titulo, medios, score_importancia, imagen_url	Abre /eventos/{id}
articulo	id, titulo, url, medio, tono, imagen_url	Abre URL original (nueva pestaña)

```
1 {
2   "respuesta": "En los últimos días, el debate sobre...",
3   "tools_usadas": ["get_eventos", "buscar_noticias"],
4   "cards": [
5     {
6       "tipo": "evento",
7       "id": 42,
8       "titulo": "Reforma tributaria en debate",
9       "medios": ["El Deber", "Los Tiempos", "Brujula Digital"],
10      "score_importancia": 0.5,
11      "imagen_url": "https://..."
12    },
13    {
14      "tipo": "articulo",
15      "id": 301,
16      "titulo": "Senado aprueba reforma en segunda lectura",
17      "url": "https://eldeber.com.bo/...",
18      "medio": "El Deber",
19      "tono": "negativo"
20    }
21  ]
22 }
```

Listing 13: Estructura de respuesta del endpoint /chat

8. Frontend (Astro + React)

La interfaz web está construida con **Astro** en modo de renderizado en servidor (SSR). El enfoque de *islas* de Astro permite que la mayor parte del HTML se genere en el servidor sin enviar JavaScript al cliente, mientras que React solo se carga para los componentes que requieren interactividad real (filtros, gráficos, chat). El estilo visual imita la tipografía y composición de un periódico clásico.

8.1. Portada (/)

La portada es la vista principal del sistema. Está implementada como un componente React interactivo (`PortadaContenido.tsx`) que recibe todos los eventos del servidor y maneja el estado de filtrado en el cliente sin recargar la página.

Filtro de categorías

En la parte superior de la portada aparece una fila de botones con las categorías detectadas automáticamente a partir de las palabras clave de los eventos (Política, Economía, Deportes, Sociedad, Internacional, Cultura). Al hacer clic en una categoría, la vista se actualiza instantáneamente mostrando solo los eventos de esa categoría. Un botón “Todas” restablece la vista completa.

Las categorías se infieren en el frontend a partir del campo `temas` de cada evento, sin requerir ningún cambio en la base de datos: el módulo `categorias.ts` aplica un diccionario de palabras clave asociadas a cada categoría.

Evento destacado

El primer evento de la categoría activa (o el más importante globalmente) se presenta en formato destacado: una tarjeta horizontal que ocupa todo el ancho disponible, con la imagen de portada a la izquierda y el texto a la derecha. Si la imagen no está disponible o no carga, la tarjeta ocupa el ancho completo con solo el texto.

Grid de eventos

El resto de los eventos se presenta en un grid de tres columnas adaptable al ancho de pantalla (*responsive*). Cada tarjeta (`EventoCard`) muestra: imagen de portada (si existe), categoría con su color, título, barra de importancia y los medios que lo cubrieron. Si la imagen falla al cargar, se muestra una franja de color correspondiente a la categoría del evento.

Asistente IA flotante

En la esquina inferior derecha de toda la interfaz aparece un botón flotante con el texto “★ Analista IA”. Al hacer clic, se despliega un panel de chat de 22×30 cm que permite al usuario hacer preguntas en lenguaje natural sobre las noticias monitoreadas. El panel se puede cerrar en cualquier momento sin perder el historial de la conversación.

Cuando el asistente responde, el texto de la respuesta aparece acompañado de tarjetas de eventos o artículos relacionados con la pregunta. Cada tarjeta es un enlace directo al evento o al artículo original, con el título, el medio y (cuando está disponible) la imagen de portada.

8.2. Cronología (/cronologia)

Esta vista reemplaza las páginas `/timeline` y `/serie` del sistema anterior, unificándolas en una visualización más informativa y visualmente clara.

Gráfico *beeswarm*

El elemento central es un gráfico SVG de tipo **beeswarm**: cada punto representa un evento, su posición horizontal indica la fecha de detección, y su posición vertical se calcula automáticamente para evitar que los puntos se superpongan (de ahí el nombre “beeswarm” — panal de abejas).

El gráfico codifica dos dimensiones de información:

- **Color**: representa el sesgo promedio del evento entre sus medios.
 - Rojo (`#8b1a1a`): sesgo promedio negativo (cobertura crítica).
 - Gris neutro (`#8a7a62`): sesgo próximo a cero (cobertura equilibrada).
 - Verde (`#1a4a1a`): sesgo promedio positivo (cobertura favorable).
 - Gris oscuro (`#4a4a4a`): sin datos de sesgo (el artículo aún no fue analizado).

La transición entre colores no es abrupta: el componente `lerpHex()` interpola el color de forma suave entre los extremos rojo y verde pasando por el neutro.

- **Tamaño**: proporcional al `score_importancia` del evento. Un evento cubierto por los seis medios (1.0) tiene un radio máximo; uno cubierto por dos medios (0.33) tiene un radio mínimo.

Al pasar el cursor sobre un punto aparece un *tooltip* con el título del evento, la fecha, el sesgo promedio y el número de medios. Al hacer clic se navega al detalle del evento.

Leyenda

Bajo el gráfico se muestra una leyenda con dos elementos: una barra de gradiente horizontal que ilustra la escala de colores de sesgo (rojo→gris→verde), y tres círculos de ejemplo que muestran la correspondencia entre tamaño y porcentaje de importancia.

Tarjetas de eventos

Debajo del gráfico se listan todos los eventos en formato de tarjeta, ordenados por fecha. El borde izquierdo de cada tarjeta tiene el color del sesgo promedio del evento, creando coherencia visual con el gráfico superior.

8.3. Sesgos (/sesgos)

Visualización centrada en las diferencias editoriales. Para cada evento con análisis de sesgo, se representa un eje horizontal de -1 a $+1$ con un punto por cada medio que lo cubre. La dispersión visual de los puntos muestra intuitivamente la divergencia: puntos

muy separados indican coberturas muy distintas, puntos agrupados indican consenso editorial. Un algoritmo de *beeswarm* evita la superposición de etiquetas cuando varios medios tienen valores similares.

8.4. Stack tecnológico

Cuadro 5: Stack del frontend

Tecnología	Rol
Astro v6 (modo SSR)	Framework principal, renderizado en servidor
React	Componentes interactivos (islas)
Tailwind CSS v4	Estilos utilitarios
Playfair Display + IM Fell English	Tipografía con estética de periódico
@astrojs/node	Adaptador para despliegue en Node.js

La arquitectura de *islas* de Astro permite que la mayor parte del HTML se genere en el servidor (sin JavaScript al cliente), mientras que React solo se carga para los componentes que requieren interactividad real (chat, filtros, gráficos SVG).

9. Decisiones de Diseño

10. Despliegue

El sistema corre en un servidor VPS de OVHcloud gestionado con **Dokploy**, una plataforma PaaS autoalojada que simplifica el despliegue de contenedores Docker con gestión automática de certificados SSL vía Traefik.

Servicios Docker

- **Backend:** imagen construida desde `Dockerfile.backend` (`python:3.11-slim`), ejecuta `uvicorn` en el puerto 8000.
- **Frontend:** imagen construida desde `Dockerfile.frontend` (`node:22-slim`, compilación multietapa), sirve la aplicación Astro en el puerto 4321.

Variables de entorno

- `DATABASE_URL`: cadena de conexión al *Session Pooler* de Supabase (PostgreSQL en la nube).
- `GEMINI_API_KEY`: clave de la API de Google AI.
- `PUBLIC_API_URL`: URL pública del backend, inyectada en el build del frontend.

Flujo de peticiones

Navegador → Traefik (HTTPS/SSL) → Astro Node.js (SSR) → FastAPI → PostgreSQL (Supabase cloud)

Las peticiones de scraping salientes son completamente independientes de Traefik: el scheduler del backend realiza peticiones HTTP directamente a los portales de noticias sin pasar por el proxy inverso.

11. Resultados y Ejemplos

A continuación se presenta un ejemplo concreto de funcionamiento del sistema con datos reales.

Evento: partido de fútbol

El evento “Bolívar empata con Nacional Potosí” fue detectado con cobertura de cuatro medios (Red Uno, El Deber, Los Tiempos, Erbol), obteniendo un *score* de importancia de $4/6 = 0.67$ (67%). El sistema detectó este evento porque cuatro redacciones distintas publicaron artículos similares el mismo día; al superar el requisito de 2+ medios, se creó el evento automáticamente.

Resumen comparativo generado por el sistema

El partido entre Bolívar y Nacional Potosí fue cubierto por cuatro medios con enfoques distintos. Brújula Digital destacó los fallos defensivos del conjunto paceño, mientras que Los Tiempos mantuvo un tono descriptivo sin valoraciones. El Deber mencionó brevemente el resultado sin análisis adicional. La divergencia de 0.40 indica diferencias moderadas en el encuadre pero sin posiciones editoriales extremas.

En el gráfico de cronología, este evento aparecería como un punto de tamaño mediano (67% de importancia) con un color tirando a gris neutro (sesgo promedio cercano a -0.17), ubicado en la fecha de detección correspondiente.

12. Limitaciones y Trabajo Futuro

Limitaciones actuales

- **Disponibilidad de imágenes:** no todos los portales bolivianos publican el metadato `og:image` en sus artículos. Los eventos sin imagen se muestran con una franja de color como alternativa, pero la experiencia visual es menos rica. Esto depende de la configuración interna de cada portal y escapa al control del sistema.

- **La Razón solo por RSS:** al ser una SPA con JavaScript, no es posible hacer scraping directo del portal. El RSS introduce el retraso típico de este medio (generalmente horas, no semanas, pero inferior al scraping directo). Además, el RSS no incluye metadatos `og:image`, por lo que los artículos de La Razón raramente tienen imagen.
- **Union-Find sin corrección:** una vez que un artículo se asigna a un evento, no puede ser reasignado. Los clusters son permanentes; no existe mecanismo de corrección automática si dos eventos distintos se agrupan erróneamente.
- **Baja cobertura cruzada:** la fragmentación regional de los medios bolivianos hace que muchos eventos solo sean cubiertos por uno o dos medios. El requisito de 2+ medios filtra correctamente estos casos, pero reduce el número total de eventos detectados. El análisis de sesgo comparativo requiere mínimo dos medios con texto.
- **Cronología de sesgo:** el gráfico beeswarm solo es informativo para eventos con análisis de sesgo ya completado. Eventos recientes aún en cola de análisis aparecen con color gris neutro, lo que puede confundirse con sesgo real de cero.
- **Velocidad de embeddings:** en CPU, agrupar 100 artículos tarda aproximadamente 70 segundos. Con una base de datos de miles de artículos, el ciclo de análisis puede extenderse varios minutos.

Trabajo futuro

- **Reconocimiento de entidades nombradas (NER):** identificar personas, organizaciones y lugares mencionados en los artículos para rastrear cómo distintos medios referencian a los mismos actores a lo largo del tiempo.
- **Resúmenes automáticos de eventos:** generar un resumen unificado que combine la perspectiva de todos los medios que cubrieron el mismo evento, ofreciendo una visión sintética al lector.
- **Notificaciones push:** alertar a usuarios suscritos cuando se detecten eventos de alta importancia ($\text{score} > 0.8$) o divergencias editoriales elevadas (divergencia > 0.7).
- **Tendencia de sentimiento por medio:** gráfico temporal que muestre si el tono editorial de cada medio hacia ciertos temas ha cambiado a lo largo de semanas o meses.
- **Expansión a más medios:** incorporar portales adicionales como Página Siete, Correo del Sur (Sucre) o ANF (Agencia de Noticias Fides) para aumentar la cobertura geográfica y temática.

Cuadro 6: Decisiones de diseño y alternativas descartadas

Decisión adoptada	Alternativa descartada	Razón
Embeddings locales (MiniLM) para clustering	Embeddings de Gemini	Coste cero en CPU; Gemini se reserva para análisis editorial que requiere comprensión profunda.
PostgreSQL + campo JSON para análisis	MongoDB	Un solo servicio (Supabase); consultas SQL para campos relacionales + flexibilidad JSON para campo variable.
Umbral coseno 0.65	0.75	Con 0.75 se perdían eventos cross-media; con 0.65 se capturan sin agrupar no relacionados.
Astro SSR + @astrojs/node	Next.js / SvelteKit	Componentes de islas (React solo donde hay interactividad), build simplificado.
APScheduler en proceso	Celery + Redis	Proyecto monolítico simple; sin dependencias extra de infraestructura.
Portal scraping sobre RSS	RSS puro	RSS de medios bolivianos: hasta 2–3 semanas de retraso; scraping es tiempo real.
Ciclo diario (CronTrigger) en lugar de cada 30/60 min	Interval scheduler	El ciclo cada 30 min agotaba la cuota de Gemini y generaba artículos sin analizar acumulados. Un ciclo diario es predecible y compatible con la cuota gratuita.
2+ medios distintos para crear un evento	Cualquier cluster	Sin este filtro, la BD se llena de artículos individuales sin posibilidad de análisis comparativo, que es el objetivo central del sistema.
og:image desde metadata Open Graph	Subir imágenes propias / sin imágenes	Los portales ya generan esta imagen para redes sociales; reutilizarla es gratis y mantiene la imagen actualizada sin infraestructura adicional.
Chat responde con tarjetas estructuradas	Solo texto plano	Las tarjetas permiten navegación directa al evento o artículo relevante, convirtiendo el chat en un punto de entrada a todo el sistema.

Cuadro 7: Análisis de sesgo por medio para el evento de fútbol

Medio	Sesgo	Interpretación
Brújula Digital	-0.40	Encuadre crítico (énfasis en errores del equipo)
El Deber	-0.10	Ligeramente negativo
Los Tiempos	+0.00	Neutral / equilibrado
Divergencia	0.40	max – min = 0.00 – (-0.40)